

Torus 网格切割项目：算法实现详解（现行版本）

2026 年 8 月 11 日

目录

1	概述	2
2	Torus 参数化生成	3
2.1	数学公式	3
2.2	三种网格来源	3
3	半边数据结构 (Half-Edge / DCEL)	3
3.1	核心数据结构	4
3.2	从三角形 / 四边形构造半边网格	4
3.3	关键操作	4
3.3.1	split_edge	4
3.3.2	split_face	5
4	切割曲线定义	5
4.1	U/V 网格线	5
4.2	Torus Knot 曲线	5
4.3	UV 域与周期性	5
5	三角形/多边形与切割线的相交检测	5
5.1	UV 展开工具（核心）	5
5.2	分类与交点（U-线为例）	6
5.3	退化情况	6
6	网格切割算法	7
6.1	cut_mesh_by_grid: U/V 交替收敛	7
6.2	为什么先 U 后 V（顺序无关性）	7

7	patch 分配	8
7.1	区域查找: <code>find_region_index</code>	8
7.2	<code>assign_patch_indices</code>	8
8	渲染管线	8
8.1	基于 <code>wgpu</code> 的渲染架构	8
8.2	摄像机	9
9	数值鲁棒性	9
9.1	容差策略	9
9.2	环面拓扑的包裹处理	9
10	文件导出	10
10.1	导出格式	10
10.2	OBJ 格式 (<code>obj.rs</code>)	10
10.3	STL 格式 (<code>stl.rs</code>)	10
10.4	PLY 格式 (<code>ply.rs</code>)	10
11	总结	10

1 概述

本项目实现了一个交互式三维 Torus（环面）网格切割工具。支持三种网格来源：

- **Quad 网格**（默认）：由 `torus::generate_unfolded_quad_mesh` 直接生成的规则四边形网格；
- **Delaunay 网格**：泊松盘采样 + 周期 Delaunay 三角化(`delaunay::generate_delaunay_mesh`)；
- **OBJ 导入**：由 `obj_loader::load_obj_as_half_edge` 解析外部网格。

在其表面生成参数化切割曲线（U/V 网格线或 Torus Knot 曲线），沿曲线把每个面切开，得到若干独立的小模型（patch），以不同颜色渲染，并支持导出为 **OBJ / STL / PLY** 三种格式。

核心技术挑战在于半边数据结构（Half-Edge / DCEL）上的切割操作，需要处理大量边界情况和数值鲁棒性问题。

与旧文档的要点差异（已在正文中逐节纠正）：

- 导出格式为 **OBJ / STL / PLY**，没有 OFF 导出；
- 相交检测用 `normalize_uv + unwrap_angle` 展开 UV，不是“枚举 5 个 `du` 偏移”；

- patch 编号用 `circular_centroid + find_region_index` (对排序后的切割值二分查找), 不是 $\lfloor \bar{u}/w \rfloor \bmod N$;
- “在 knot 线上” 的容差为 `MERGE_EPS = 1e-4`, 不是 10^{-8} ;
- `cut_mesh_by_grid` 采用 U/V 交替迭代直到面数稳定 (通常 2-3 轮), 并有 `MAX_FACES=150000` 硬上限保护。

2 Torus 参数化生成

2.1 数学公式

Torus 的参数方程由大半径 R (环心到管心的距离) 和小半径 r (管道半径) 定义:

$$\begin{aligned} x(u, v) &= (R + r \cos v) \cos u \\ y(u, v) &= (R + r \cos v) \sin u \\ z(u, v) &= r \sin v \end{aligned} \tag{1}$$

其中 $u \in [0, 2\pi)$ 为绕大圆的参数角, $v \in [0, 2\pi)$ 为绕管道的参数角。详见 `torus.rs` (`torus.tex`)。

2.2 三种网格来源

- **Quad:** `generate_unfolded_quad_mesh(R, r, res_u, res_v)` 返回 $(res_u+1) \times (res_v+1)$ 个顶点与 $res_u \times res_v$ 个四边形 (不去回绕)。`HalfEdgeMesh::from_quads` 构造半边网格。
- **Delaunay:** `generate_delaunay_mesh(R, r, n_points, seed)` 返回全三角形网格; 拓扑 (UV/三角形/半边) 与 R, r 无关, 拖动 R, r 时只重映射顶点 3D 位置 (`torus_position`), 故 UI 中缓存 `delaunay_base` 以降低 rebuild 延迟。`HalfEdgeMesh::from_tri` 构造。
- **OBJ:** `load_obj_as_half_edge(path)` 解析 v/vt/f, 多边形 fan 三角化, 按 (pos, uv) 去重; 若 OBJ 带 UV 且拟合为 Torus, 则 `remap_uvs_from_analytical_model` 用 `SurfaceModel::compute_uv` 重写每点 UV, 使切割在一致参数域上进行。

默认 `resolution_u=40`, `resolution_v=24` (Quad), Delaunay 默认 `delaunay_points=500`, 均可在 UI 中调整——参数化分辨率由用户配置决定, 不是固定的 41×25 (旧文档数字已过时)。

3 半边数据结构 (Half-Edge / DCEL)

半边数据结构是网格切割操作的基础。每一个无向边被拆分为两条方向相反的半边 (half-edge)，每条半边属于一个面 (face)，并指向下一条半边 (next)、上一条半边 (prev) 以及反向的孪生半边 (twin)。

3.1 核心数据结构

```
1 pub struct HEVertex {
2     pub position: Vec3,    // 三维位置
3     pub uv: Vec2,          // 参数坐标 (u, v)
4     pub outgoing: HalfEdgeId, // 从该顶点出发的一条半边
5 }
6
7 pub struct HalfEdge {
8     pub origin: VertexId, // 起始顶点
9     pub twin: HalfEdgeId, // 反向半边 (无向边的另一半)
10    pub next: HalfEdgeId, // 沿面逆时针的下一条半边
11    pub prev: HalfEdgeId, // 沿面逆时针的上一条半边
12    pub face: FaceId,     // 所属面
13 }
14
15 pub struct HEFace {
16     pub half_edge: HalfEdgeId, // 该面的任意一条半边
17     pub valid: bool,           // 切割后是否仍有效
18     pub patch_index: Option<(usize, usize)>, // 所属patch编号
19 }
```

3.2 从三角形 / 四边形构造半边网格

给定顶点位置数组、UV 数组和三角形索引三元组 (或四边形索引四元组)，构造过程如下：

1. 为每个面创建相应的半边。
2. 为每个顶点记录一条出射半边。
3. 利用哈希表 (v_a, v_b) ($v_a < v_b$) 查找并配对孪生半边。

`from_triangles` 用于 Delaunay/OBJ, `from_quads` 用于 Quad。

3.3 关键操作

3.3.1 split_edge

在一条半边上插入一个新顶点 V_{new} ，将原半边及其孪生分别拆分为两段，并更新 next/prev/twin/origin/face 等指针。

3.3.2 split_face

在面的边界上两个顶点 v_a 和 v_b 之间添加一条对角线，将一个面拆分为两个：创建对角线半边及其反向半边，重连周围的 next/prev 指针，重新分配两个面的 face 归属。

4 切割曲线定义

4.1 U/V 网格线

U-常值线： $u = \text{const}$ ，沿 v 方向延伸；V-常值线： $v = \text{const}$ ，沿 u 方向延伸。它们的 UV 位置由 UI 的 loop 参数决定：

- **Quad 模式：**线位置吸附到最近的网格顶点线 ($u = 2\pi \cdot k / \text{res}_u$)，使切割沿面边界走、不穿过面内部，从而保持四边形面型；
- **Delaunay/OBJ 模式：**取单元中心等分 ($u = 2\pi \cdot (i + 0.5) / n$)，会穿过面内部，切割后由三角化保持全三角形。

4.2 Torus Knot 曲线

Knot 曲线 $v = ku$ 在参数方域内有 k 条分支 ($v = ku + 2\pi n$)。支持两条 knot (k_1, k_2) 独立开关。详见 knot_algorithm.tex。

4.3 UV 域与周期性

环面在 u 和 v 方向周期 2π 。网格生成时 $u = 0$ 与 $u = 2\pi$ 是独立顶点（不回绕），相交检测通过 normalize_uv+unwrap_angle 在局部展开，避免接缝误判。

5 三角形/多边形与切割线的相交检测

5.1 UV 展开工具（核心）

相交检测不再用“枚举 5 个 du 偏移”，而是：

1. `normalize_uv(val, min, max) = (val-min)/(max-min) * 2π` $UV [min,max] [0,2\pi)$, 使 `unwrap_angle` (周期 2π) 能正确工作。
2. `unwrap_angle(angle, reference)`: 通过反复 $\pm 2\pi$ 使 $angle - reference \in [-\pi, \pi]$, 得到相对参考点的连续坐标, 正确处理接缝。

5.2 分类与交点 (U-线为例)

`intersect_face_u(face_verts, u_const, uv_range):`

1. 对所有顶点, 先 `normalize_uv` 再 `unwrap_angle` (相对首顶点) 得到连续 (a_i, b_i) ; 对 `u_const` 同样处理得到 `uc_unwrap`。
2. 按 $diff = a_i - uc_unwrap$ 分类:

$$side_i = \begin{cases} \text{OnLine} & |diff| < \text{MERGE_EPS} \\ \text{Less} & diff < 0 \\ \text{Greater} & diff > 0 \end{cases}$$

3. 若只有同侧 (无 Less/Greater 且无 OnLine) $\rightarrow$ 无交点; 若既非两侧跨线又无 OnLine $\rightarrow$ 返回 None。
4. 对每条边 $(i0, i1)$, 跳过两端皆 OnLine、或两端同侧且无 OnLine 的边; $du = a[i1] - a[i0]$, 若 $|du| \leq \text{EPSILON}$ 跳过 (平行于线);

$$t = (uc_unwrap - a_{i0})/du, \quad \text{夹到}[0, 1]$$

并过滤起点 OnLine 且 $t < \text{MERGE_EPS}$ 、终点 OnLine 且 $t > 1 - \text{MERGE_EPS}$ 的情形。

5. 交点 UV 由 `clamp_uv` 夹回 $[0, 2\pi]$, 3D 坐标按边线性插值。

`intersect_face_v` 完全对称 (交换 $u \leftrightarrow v$)。

5.3 退化情况

- 顶点在线上 ($t \approx 0$ 或 $t \approx 1$): 加入 `on_line_vertices`。
- 边平行于线 ($|du| < \text{EPSILON}$): 无交点。
- 整条边在线上 (两端 OnLine): 不切。
- 接缝面: 经 `unwrap_angle` 展开后, 短路径边不会被误判为跨过 2π 。

6 网格切割算法

6.1 cut_mesh_by_grid: U/V 交替收敛

Input: 半边网格 M , U 切割值集合 U , V 切割值集合 V , UV 域 R

Output: 切割后的网格 M , 面保持原面型 (Quad 保持四边形 / Delaunay 保持全三角形)

for 最多 6 轮 **do**

 记录切割前面数 $N_{before} \leftarrow M.num_valid_faces()$;

for 每条 $u \in U$ **do**

 | cut_mesh_by_u_line(M, u, R)

end

for 每条 $v \in V$ **do**

 | cut_mesh_by_v_line(M, v, R)

end

if $M.num_valid_faces() == N_{before}$ **then**

 | 跳出 (已收敛)

end

if $M.num_valid_faces() > 150000$ **then**

 | 记录警告并跳出 (防指数增长)

end

end

if *finalize_triangles* **then**

 | 把残留 > 3 边多边形 fan 三角化

end

验证 $M.validate()$ 并打印最终面数/顶点数;

为什么交替迭代: 一条切割线切出的新面可能仍跨另一条切割线 (例如 U 线切完后, V 线切分产生的多边形仍可能跨 U 线), 需反复执行全部切割线直到面数不再变化——通常 2-3 轮收敛。每轮实时检查面数变化即决定是否停止。

面型保持: 相交的非三角形面在切割处**局部三角化** (cut_face_local), 其余面保持原面型——Quad 网格的四边形与 Knot 远处区域都不受影响。Delaunay 模式传入 *finalize_triangles*=true, 使切割后全为三角形 (一条直线切三角形必产生四边形 + 三角形, 靠收尾三角化消除残留多边形)。

6.2 为什么先 U 后 V (顺序无关性)

当前实现在一轮中依次处理全部 U 线再全部 V 线, 但关键点在于“交替迭代直到收敛”: 由于循环条件实时检查面数且重复整轮, 最终结果与 U/V 处理的先后顺序无关, 所有交叉都会被正确切开。

7 patch 分配

切割完成后,为每个有效面计算其参数质心并归属区域。**注意:**这里使用 `circular_centroid` (先相对首顶点 `unwrap`, 再取算术均值, 最后映射回 $[min, max]$ 原 UV 空间), 不再使用旧文档的“圆均值/算术均值之争”, 也不用 $\lfloor \bar{u}/w \rfloor \bmod N$ 公式。

7.1 区域查找: `find_region_index`

将切割值排序后, 用线性/二分查找确定质心落在第几段:

```
1 fn find_region_index(value: f64, sorted_cuts: &[f64]) -> usize {
2     if sorted_cuts.is_empty() { return 0; }
3     if value < sorted_cuts[0] { return sorted_cuts.len() - 1; }
4     for i in 0..sorted_cuts.len()-1 {
5         if value < sorted_cuts[i+1] { return i; }
6     }
7     sorted_cuts.len() - 1
8 }
```

7.2 `assign_patch_indices`

1. 对 U/V 切割值分别排序, 并用 `normalize_uv` 归一化到 $[0, 2\pi)$ 。
2. 对每个有效面, `circular_centroid` 得到 $(\bar{u}, \bar{v})$, 再归一化。
3. $pu = \text{find_region_index}(\bar{u}_{\text{norm}}, \text{sorted_}u), pv = \text{find_region_index}(\bar{v}_{\text{norm}}, \text{sorted_}v)$ 。
4. 写入 `face.patch_index = Some((pu, pv))`。

周期语义: n 条切割线把环域分成 n 段 (值小于第一条线者归入最后一段), 故 `cut_grid_dims` 返回 (n_u, n_v) (无切割时为 $(1, 1)$)。

Knot 模式则使用 `assign_multi_knot_patch_indices` (见 `knot_algorithm.tex`)。

8 渲染管线

8.1 基于 `wgpu` 的渲染架构

1. **Mesh Pass:** 顶点着色器接收 `position`、`normal`、`color`、`uv`、`material`, 经 MVP 变换后输出; 片元着色器支持 42 种着色模式 (PBR/Matcap/Toon/…), 含方向光 + 环境光 + 三点光照。

2. **Edge Pass**（线框/切割线）：把网格边与切割线作为彩色线段单独绘制，叠加在 Mesh Pass 之上。
3. **egui Overlay Pass**：即时模式 GUI 在最上层绘制（浮于 3D 场景之上，不清屏）。

所有 Pass 只渲染到中央 3D 视口区域（左右面板不被场景覆盖）。

8.2 摄像机

球坐标系轨道摄像机：

$$\text{eye} = \begin{pmatrix} \rho \cos \phi \cos \theta \\ \rho \sin \phi \\ \rho \cos \phi \sin \theta \end{pmatrix} + \text{target} \quad (2)$$

θ 方位角（鼠标水平拖动）， ϕ 极角（鼠标垂直拖动）， ρ 距离（滚轮）；WASD/QE 平移 target；左键旋转、右键/中键平移。

9 数值鲁棒性

9.1 容差策略

- **EPSILON = 1e-8**：用于“边是否平行于切割线”（ $|\text{du}| \leq \text{EPSILON}$ 即判定平行）以及交点参数 t 的边界判定。
- **MERGE_EPS = 1e-4**：用于“顶点是否在切割线上”的判定（ $|\text{diff}| < \text{MERGE_EPS}$ 即 OnLine），以及跳过与 OnLine 顶点重合的交点。因相邻边分裂产生的顶点经 unwrap+wrap 往返后 ϕ 可能略超 10^{-8} ，故用更宽松的 10^{-4} 。
- 插值参数 t 统一夹到 $[0, 1]$ 。

旧文档把 OnLine 容差写成 10^{-8} ，与代码不符——实际是 **MERGE_EPS=1e-4**。

9.2 环面拓扑的包裹处理

- UV 经 `normalize_uv` 归一化到 $[0, 2\pi)$ 后再 `unwrap_angle` 展开。
- 接缝处（ $u \approx 0$ 或 $u \approx 2\pi$ ）的三角形通过相对首顶点的连续展开正确判定跨线。
- 交点 UV 用 `clamp_uv` 夹回，不回绕。

10 文件导出

10.1 导出格式

当前版本支持三种格式 (ExportFormat 枚举), 没有 OFF 导出:

```
1 pub enum ExportFormat { Obj, Stl, Ply }
```

- `export_mesh(mesh, path, format)`: 按格式导出整张网格 (所见即所得——UV 平面视图导出展开坐标, 3D 视图导出环面坐标)。
- `export_by_patch(mesh, dir, format)`: 按 patch 批量导出, 每补片一个文件 `<dir>/patch_<pu>_<pv>.<ext>`。

10.2 OBJ 格式 (obj.rs)

按 patch 分组, 使用 `g patch_<pu>_<pv>` 组标签:

```
g patch_0_0
f v/vt//vn v/vt//vn v/vt//vn
...
g patch_0_1
...
```

顶点法向量通过相邻面法向量面积加权平均计算 (`compute_vertex_normals`)。

10.3 STL 格式 (stl.rs)

ASCII STL, 每面写法向量 (右手定则) 与三个顶点, 包裹在 `solid torus_grid_cutter` / `endsolid` 之间。

10.4 PLY 格式 (ply.rs)

ASCII PLY, 头部声明 `element vertex` / `element face`, 每面写 `3 i0 i1 i2`。

11 总结

本项目的核心难点在于半边数据结构上的切割操作。关键点:

1. 半边数据结构的精确操作: `split_edge` 和 `split_face` 需要更新多个指针字段, 任何一个错误都会导致网格拓扑损坏。

2. **UV 展开驱动相交:** 用 `normalize_uv+unwrap_angle` 在局部连续展开, 正确处理环面周期性与接缝, 比旧“枚举多路径 du”更稳健。
3. **U/V 交替收敛:** 反复整轮切割直到面数稳定, 保证所有交叉被切开; `MAX_FACES=150000` 上限防止极端参数下指数增长。
4. **面型保持:** 切割处局部三角化, Quad 四边形与 Knot 远处面不受影响。
5. **数值鲁棒性:** `EPSILON=1e-8` 判平行, `MERGE_EPS=1e-4` 判在线上, 覆盖顶点在线上、边近似平行、交点贴近顶点等退化情形。
6. **导出 OBJ/STL/PLY:** 按 patch 分组, 所见即所得。